

파이썬으로 배우는 완전탐색

Brute Force — 모든 경우를 확인해서 답 찾기

CodeWithCathy

완전탐색이란?

가능한 경우를 하나도 빠짐없이 전부 확인해서 답을 찾는 방법

Brute Force라고도 불러요. 똑똑한 규칙을 찾기 전에 "다 해보면 되지 않을까?"를 먼저 따져보는 거예요.

완전탐색의 핵심 한 단어

경우의 수 세기

✓

장점

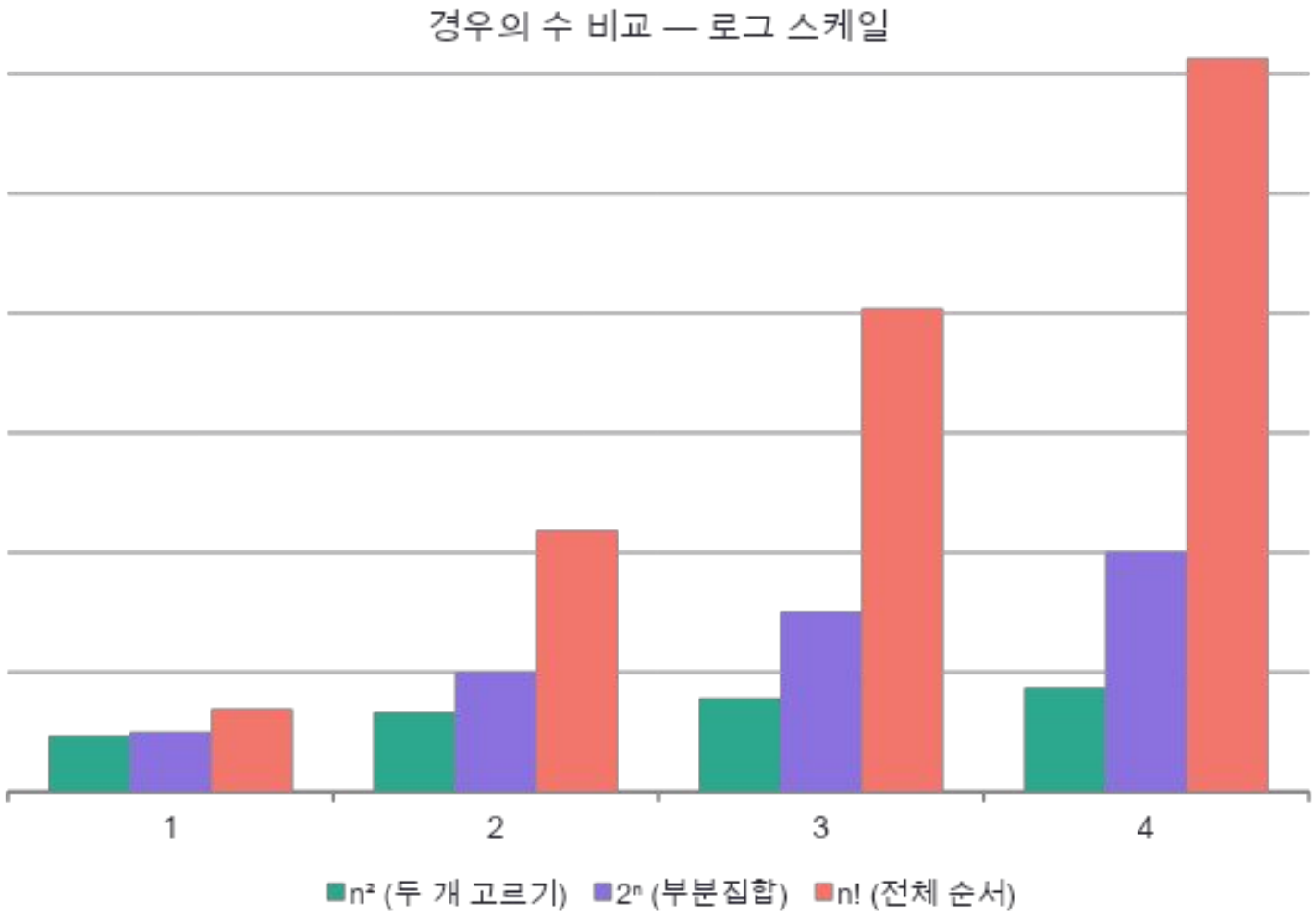
모든 경우를 보니까 답을 놓치지 않아요
구현이 단순해서 실수가 적어요

!

단점

경우의 수가 많으면 시간 초과가 나요
그래서 먼저 경우의 수를 계산해야 해요

언제 쓸 수 있을까? 경우의 수 계산하기



기준: 파이썬은 1초에 약 1천만 번

약 100만

n=20의 부분집합 → 가능

약 2.4×10^{18}

n=20의 전체 순서 → 불가능

제한사항을 보고 먼저 계산하세요

완전탐색 구현 도구 4가지

먼저 이렇게 물어보세요: “무엇을 전부 만들어서 확인해야 하지?”

정해진 범위의 숫자를 하나씩 넣어보면 된다	→ **① 반복문**	예) 1~50 중 자릿수 합이 10인 수
몇 개를 고르거나, 순서대로 줄 세운 경우를 다 봐야 한다	→ **② itertools**	예) 카드 두 장 고르기, 세 자리 수 만들기
하나씩 고르다가 조건이 깨지면 중간에 그만둔다	→ **③ 재귀 / 백트래킹**	예) 합이 3 이하인 부분집합
연결된 칸·장소를 따라가며 전부 방문한다	→ **④ BFS / DFS**	예) 미로에서 갈 수 있는 칸 세기

도구 ① 반복문: 범위 안의 값을 하나씩 대입

언제? 후보가 “1부터 n까지”처럼 범위로 정해져 있을 때

어떻게 동작하나요?

1. 범위 안의 수를 처음부터 끝까지 하나씩 꺼내요
2. 꺼낸 수가 조건에 맞는지 확인해요
3. 맞으면 답에 담거나 개수를 세요

범위가 두 개면 for문을 두 번 겹쳐요 (예: 가로 × 세로)

Python

```python
# 1~50 중 각 자리 숫자의 합이 10인 수
answer = []
for n in range(1, 51):
    if sum(map(int, str(n))) == 10:
        answer.append(n)
print(answer)
```

실행 결과: 50개를 전부 확인해서 4개를 찾음

19 **28** **37** **46**

경우의 수 = 범위의 크기 (50)

도구 ② itertools: 고르고 줄 세우기

Python

```python
from itertools import permutations, combinations, product

items = ["A", "B", "C"]

list(permutations(items, 2))
# AB AC BA BC CA CB  → 6개

list(combinations(items, 2))
# AB AC BC  → 3개

list(product(items, repeat=2))
# AA AB AC BA BB BC CA CB CC  → 9개
```

permutations

순열 · 순서 O, 중복 X

nPr

combinations

조합 · 순서 X, 중복 X

nCr

product

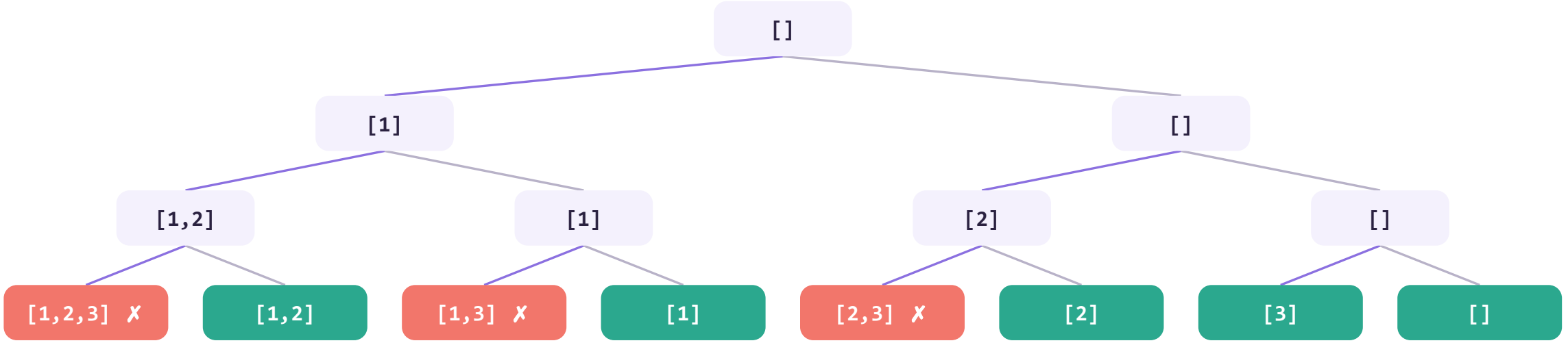
중복순열 · 순서 O, 중복 O

n^r

판단 기준: 순서가 바뀌면 다른 경우인가? → 순열 / 같은 경우인가? → 조합

도구 ③ 재귀 / 백트래킹: 고르다가 안 되면 되돌아가기

[1, 2, 3]에서 합이 3 이하인 부분집합 찾기 — 숫자마다 “넣는다 / 안 넣는다”를 차례로 결정



Python

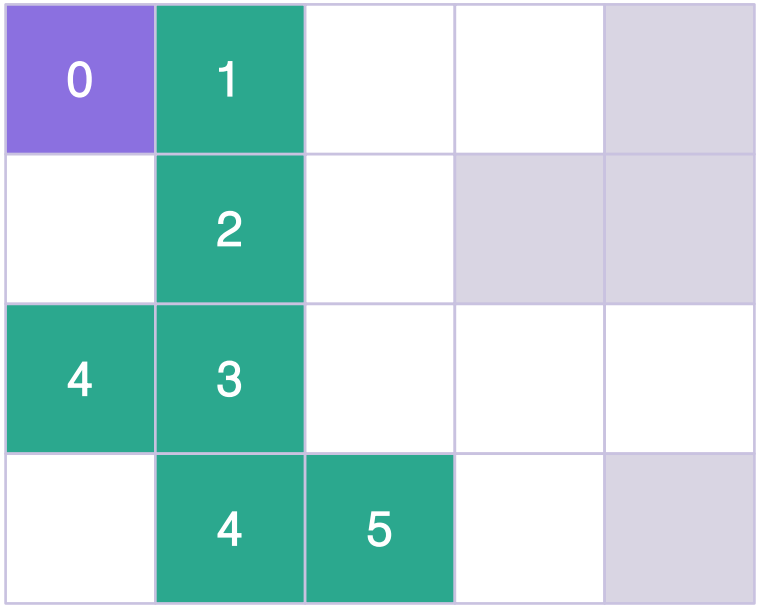
```python
def dfs(i, picked, total):
    if total > 3: return          # 가지치기
    if i == len(nums):
        result.append(picked); return
    dfs(i + 1, picked + [nums[i]], total + nums[i])  # 넣기
    dfs(i + 1, picked, total)                        # 안 넣기
```

보라색 선 = 넣기, 회색 선 = 안 넣기
빨간 ✗ = 합이 3을 넘어서 버림

조건이 깨지면 더 내려가지 않고 바로 돌아오는 것을 백트래킹(가지치기)이라고 해요

도구 ④ BFS / DFS: 연결된 곳을 따라가며 전부 방문

(0,0)에서 출발해 상하좌우로 1인 칸만 이동할 때, 갈 수 있는 칸은 몇 개?



숫자 = 출발점에서 몇 걸음째에 도착했는지
회색 = 땅이지만 연결이 안 돼서 못 감
흰색 = 갈 수 없는 칸(0)

정답: 7칸

Python

```python
from collections import deque

def bfs(grid):
    q = deque([(0, 0)])
    visited = {(0, 0)}
    while q:
        r, c = q.popleft()
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
                if grid[nr][nc] == 1 and (nr, nc) not in visited:
                    visited.add((nr, nc))
                    q.append((nr, nc))
    return len(visited)
```

완전탐색 문제를 푸는 4단계

1

경우의 수 계산

제한사항 최댓값으로 다 돌 수 있나?

```
8! = 40,320
→ OK
```

2

후보 정하기

무엇을 전부 만들어 볼까?

```
두 장 고르기
세 자리 수
```

3

도구 고르기

반복문? 순열? 조합?

```
combinations(
  cards, 2)
```

4

검사 & 갱신

조건 확인 후 답 업데이트

```
if 조건:
  best = max(...)
```

이제 이 4단계로 간단한 예제 두 개를 풀어볼게요

예제 ① 합이 target인 카드 쌍

문제 숫자 카드 중 서로 다른 두 장을 골라 합이 target이 되는 경우는 몇 가지?

`cards = [1, 2, 3, 4, 5],  target = 6` → 정답 2

(1, 2) 합 3	(1, 3) 합 4	(1, 4) 합 5	**(1, 5) 합 6**	(2, 3) 합 5
(2, 4) 합 6	(2, 5) 합 7	(3, 4) 합 7	(3, 5) 합 8	(4, 5) 합 9

경우의 수

5C2 = 10

카드가 1,000장이어도 약 50만 → 충분

10가지를 전부 만들어 보고, 합이 6인 것만 세기

예제 ①: 코드로 옮기기

combinations

```python
from itertools import combinations

def solution(cards, target):
    count = 0
    for a, b in combinations(cards, 2):
        if a + b == target:
            count += 1
    return count
```

이중 for문

```python
def solution(cards, target):
    count = 0
    n = len(cards)
    for i in range(n):
        for j in range(i + 1, n):
            if cards[i] + cards[j] == target:
                count += 1
    return count
```

j는 i+1부터

range(n)으로 하면 같은 카드를 두 번 쓰거나 같은 쌍을 두 번 세요

둘 다 정답

조합을 직접 만들면 이중 for문, 자동으로 만들면 combinations

예제 ② 가장 큰 세 자리 짝수 만들기

문제 숫자 카드 중 세 장을 골라 늘어놓을 때, 만들 수 있는 가장 큰 짝수는?

`digits = [1, 7, 4, 2]` → 정답 `742` (짝수가 없으면 `-1`)

174	172	147	142	127	124	714	712
741	742	721	724	417	412	471	472
421	427	217	214	271	274	241	247

짝수 짝수 중 최댓값

경우의 수

4P3 = 24

순서가 바뀌면 다른 수 → 순열

24가지를 전부 만들고, 짝수만 남겨서 가장 큰 값 고르기

예제 ②: 코드로 옮기기

Python

```python
from itertools import permutations

def solution(digits):
    best = -1    # 짝수가 없을 때의 답
    for a, b, c in permutations(digits, 3):
        num = a * 100 + b * 10 + c
        if num % 2 == 0:
            best = max(best, num)
    return best
```

1

튜플을 숫자로

(7, 4, 2) → 7×100 + 4×10 + 2 = 742

!

처음 값 정하기

best = 0으로 두면 짝수가 없을 때 −1 대신 0이 나와요

다른 방법

```
num = int(f"{a}{b}{c}")   # 문자열로 이어 붙이기
```

어떤 도구를 쓸까? 한눈에 정리

도구	언제?	경우의 수	예
이중 for문	두 개를 고를 때	n^2	예제 ① 카드 쌍
combinations	순서 상관없이 r개	nCr	예제 ① 카드 쌍
permutations	순서 있게 r개	nPr	예제 ② 세 자리 수
product	같은 것을 여러 번	n^r	비밀번호, 주사위
재귀 / DFS	조건에 따라 중간에 멈출 때	경우에 따라	다음 단원

시간 초과가 나면? 정렬 · DP · 가지치기로 확인할 경우를 줄여요

오늘의 정리

1

완전탐색은 가능한 경우를 전부 확인하는 방법

경우의 수가 작으면 가장 확실하고 실수가 적어요

2

먼저 경우의 수를 계산하기

파이썬은 1초에 약 1천만 번 · 제한사항의 최댓값으로 계산

3

도구 고르기: 반복문 · itertools · 재귀 · BFS/DFS

“무엇을 전부 만들어야 하지?”로 결정해요